RÉPUBLIQUE DU CAMEROUN
Paix – Travail – Patrie
Ministère des Enseignements Secondaires
DRES-Sud · DDES Dja-et-Lobo

REPUBLIC OF CAMEROON
Peace – Work – Fatherland
Ministry of Secondary Education
South RDSE · Dja and Lobo DDSE

Requêtes de la base de données

Toutes les lectures, ajouts, modifications et suppressions
faits par l'application de gestion scolaire du CETIC de Mezesse

Référence technique

Base de données	PostgreSQL (Neon en production)
Accès	API Spring Boot : JPA et Hibernate, migrations Flyway
Contenu	150 requêtes : 101 lectures, 19 ajouts, 17 modifications, 13 suppressions
Vérification	Chaque requête contrôlée sur la base par PostgreSQL (EXPLAIN) ; liste comparée au SQL réellement exécuté par l'API
Date	Octobre 2026

Sommaire

1. Lire ce document
2. Les tables
3. Comptes et connexion
4. Établissement et barèmes
5. Années scolaires et séquences
6. Spécialités et classes
7. Groupes de matières
8. Matières
9. Élèves, inscriptions et renvois
10. Notes et compétences visées
11. Journal des décisions
12. Notifications
13. Attestations de présence effective au poste
14. Suppressions en cascade
15. Opérations en plusieurs requêtes
16. Démarrage : migrations et amorçage

Domaine	Lire	Ajouter	Modifier	Supprimer
3. Comptes et connexion	10	1	1	1
4. Établissement et barèmes	6	2	1	1
5. Années scolaires et séquences	11	2	3	1
6. Spécialités et classes	16	2	2	2
7. Groupes de matières	11	2	1	3
8. Matières	9	1	1	1
9. Élèves, inscriptions et renvois	22	4	3	2
10. Notes et compétences visées	8	2	2	2
11. Journal des décisions	4	1	1	—
12. Notifications	2	1	2	—
13. Attestations de présence effective au poste	2	1	—	—
Total	**101**	**19**	**17**	**13**

1. Lire ce document

Seule l'API (dépôt `cetic-mezesse-backend`) parle à la base : le site web et l'application mobile passent par elle. L'API utilise **JPA** avec **Hibernate**, qui écrit le SQL à partir des entités Java et des méthodes des dépôts (`repository/`). Ce document donne, pour chaque requête, ce qu'elle fait, **quand** elle s'exécute (action dans l'application et route de l'API) et son **SQL**.

Comment lire le SQL

- **Noms entre guillemets.** Les tables et les colonnes gardent leurs majuscules (`"Classe"`, `"anneeId"`) : PostgreSQL exige alors des guillemets. Les noms tout en minuscules (`nom`, `id`) s'en passent.
- **Paramètres.** `:anneeId`, `:ids` … sont les valeurs fournies par l'application ; Hibernate les envoie comme paramètres (`?`), jamais collées dans le texte, ce qui empêche toute injection SQL. `IN (:ids)` reçoit une liste.
- **Forme simplifiée.** Le SQL est écrit pour être lu : `SELECT *` remplace la liste des colonnes qu'Hibernate énumère, et les alias sont courts. Les conditions, tris et colonnes écrites sont celles qu'envoie l'API. Pour quelques recherches par clé étrangère (compter les notes d'une matière, lire les notifications d'un compte…), Hibernate passe par une jointure sur la table liée et filtre sur son `id` au lieu de filtrer la colonne directement : le résultat est le même.
- **Identifiants.** Les identifiants (`id`) sont générés par l'application (format cuid, par exemple `cmvdtysczlku7c13ej18jpaub`), pas par la base.
- **Énumérations.** Le rôle (`PROVISEUR` pour le Directeur, `SURVEILLANT_GENERAL`, `PROFESSEUR`), le statut d'une inscription (`ACTIF`, `RENVOYE`), le sexe (`M`, `F`) et le type de barème (`GRADE`, `COTE`) sont des types énumérés PostgreSQL.

LIRE SELECT AJOUTER INSERT MODIFIER UPDATE SUPPRIMER DELETE

Ce qu'Hibernate ajoute de lui-même

- **Chargements groupés.** Quand une page affiche des objets liés (le professeur de chaque matière, l'année de chaque classe…), Hibernate les lit d'un coup, jusqu'à 100 à la fois : `SELECT * FROM "Utilisateur" WHERE id = ANY(:ids)`. Cela vaut pour toutes les tables et n'est pas répété dans chaque section.
- **Mises à jour complètes.** Un `UPDATE` réécrit toutes les colonnes de la ligne, même celles qui ne changent pas.
- **Transactions.** Chaque action de l'utilisateur s'exécute dans une transaction : toutes ses requêtes réussissent ensemble, ou aucune n'est gardée (chapitre 15).

Comment ce document a été vérifié

- Chaque requête a été soumise à PostgreSQL avec `EXPLAIN` sur une copie de la base : syntaxe, tables, colonnes et types sont corrects (150 sur 150).
- Le SQL réellement envoyé par l'API a été enregistré pendant un parcours de toutes ses routes (lectures, écritures, envois de fichiers, désapprobations) ; chaque instruction enregistrée correspond à une requête de ce document.

2. Les tables

Table	Contenu	Unicité
`Utilisateur`	Comptes : Directeur, SG, professeurs ; carrière pour les attestations	téléphone, e-mail, matricule
`Etablissement`	Une seule ligne : en-tête des documents, préfixe des matricules, seuil du tableau d'honneur	—
`Bareme`	Mentions des grades et des cotes, avec leur note minimale	(type, libellé)
`AnneeScolaire`	Années scolaires, dont une en cours	libellé
`Sequence`	Six séquences par année, saisie ouverte ou fermée	(année, numéro)
`Specialite`	Spécialités communes à toutes les années	nom
`Classe`	Classes d'une année, avec spécialité et professeur principal	(année, nom)
`GroupeMatiere`	Groupes de matières d'une année	(année, nom)
`_ClasseToGroupeMatiere`	Rattachement des groupes aux classes (A : classe, B : groupe)	(A, B)
`Matiere`	Matières d'une classe : groupe, coefficient, ordre, professeur	(classe, nom)
`Eleve`	Fiche de l'élève, photo, parent	matricule, matricule national
`Inscription`	L'élève dans une classe pour une année ; statut actif ou renvoyé	(élève, année)
`Renvoi`	Renvois et réintégrations	—
`Note`	Notes sur 20	(inscription, matière, séquence)
`CompetenceVisee`	Compétence visée d'une matière pour une séquence	(matière, séquence)
`Compteur`	Compteurs des matricules et des numéros d'attestation	clé
`Decision`	Journal des décisions du SG et de leurs désapprobations	—
`Notification`	Messages de la cloche	—
`Attestation`	Registre des attestations de présence effective au poste	référence
`flyway_schema_history`	Migrations appliquées (tenue par Flyway, voir chapitre 16)	—

`_prisma_migrations`	Historique de l'ancienne API Node ; n'est plus lue ni écrite	—

3. Comptes et connexion

Un compte par personne : le Directeur (rôle PROVISEUR dans la base), le Surveillant Général et les professeurs. Le mot de passe n'est jamais stocké en clair : la colonne motDePasse contient son empreinte bcrypt.

Tables : `Utilisateur`

Lire

LIRE R1. Trouver le compte qui se connecte

Quand : Connexion (POST /api/auth/connexion). Le même identifiant peut être un e-mail ou un téléphone.

```
SELECT * FROM "Utilisateur" u
WHERE u.email = :identifiant OR u.telephone = :identifiant
ORDER BY u."creeLe";
```

LIRE R2. Lire un compte par son identifiant

Quand : À chaque requête authentifiée (le compte du jeton doit encore être actif), puis dans la plupart des actions qui concernent une personne : Mon profil, fiche d'un enseignant, carrière, attestation, désapprobation.

```
SELECT * FROM "Utilisateur" WHERE id = :id;
```

LIRE R3. Lister tous les comptes

Quand : Liste des personnels sans filtre (GET /api/utilisateurs), par exemple pour choisir le bénéficiaire d'une attestation.

```
SELECT * FROM "Utilisateur" ORDER BY role, nom, prenom;
```

LIRE R4. Lister les comptes d'un rôle

Quand : Page Enseignants (GET /api/utilisateurs?role=PROFESSEUR) ; titre « Directeur » ou « Directrice » imprimé sur les documents (rôle PROVISEUR).

```
SELECT * FROM "Utilisateur" WHERE role = :role ORDER BY nom, prenom;
```

LIRE R5. Compter les comptes actifs d'un rôle

Quand : Statistiques de l'établissement : nombre d'enseignants.

```
SELECT count(*) FROM "Utilisateur" WHERE role = :role AND actif = true;
```

LIRE R6. Savoir si un compte de ce rôle existe

Quand : Amorçage d'une base vide : le Directeur et le SG ne sont créés qu'une fois.

```
SELECT u.id FROM "Utilisateur" u WHERE u.role = :role LIMIT 1;
```

LIRE R7. Vérifier qu'un téléphone est libre

Quand : Création d'un enseignant, modification de sa fiche, Mon profil : un téléphone ne sert qu'à un seul compte.

```
SELECT * FROM "Utilisateur" u
WHERE u.id <> :id AND u.telephone = :telephone
LIMIT 1;
```

LIRE R8. Vérifier qu'un e-mail est libre

Quand : Mêmes actions, pour l'adresse e-mail.

```
SELECT * FROM "Utilisateur" u
WHERE u.id <> :id AND u.email = :email
LIMIT 1;
```

LIRE R9. Vérifier qu'un matricule de la fonction publique est libre

Quand : Carrière d'un personnel (fiche enseignant, nouvelle attestation).

```
SELECT * FROM "Utilisateur" u
WHERE u.id <> :id AND u.matricule = :matricule
LIMIT 1;
```

LIRE R10. Professeurs actifs qui enseignent dans l'année

Quand : Ouverture d'une séquence : ces professeurs reçoivent une notification.

```
SELECT DISTINCT p.id
FROM "Matiere" m
JOIN "Classe" c ON c.id = m."classeId"
JOIN "Utilisateur" p ON p.id = m."professeurId"
WHERE c."anneeId" = :anneeId AND p.role = :role AND p.actif = true;
```

Ajouter

AJOUTER R11. Créer un compte

Quand : Nouvel enseignant (POST /api/utilisateurs), avec le mot de passe provisoire « password » ; amorçage d'une base vide (Directeur et SG).

```
INSERT INTO "Utilisateur" (id, nom, prenom, telephone, email, "motDePasse", role, actif,
  "doitChangerMotDePasse", discipline, sexe, matricule, grade, fonction, "datePriseService",
  "dateEntreeFonctionPublique", "acteNomination", "creeLe", "majLe")
VALUES (:id, :nom, :prenom, :telephone, :email, :motDePasse, :role, :actif,
  :doitChangerMotDePasse, :discipline, :sexe, :matricule, :grade, :fonction, :datePriseService,
  :dateEntreeFonctionPublique, :acteNomination, :creeLe, :majLe);
```

Modifier

MODIFIER R12. Mettre à jour un compte

Quand : Modifier la fiche d'un enseignant ou sa carrière, le désactiver ou le réactiver, réinitialiser son mot de passe ; Mon profil (dont la civilité) ; changer son mot de passe ; désapprobation qui remet l'état précédent.

```
UPDATE "Utilisateur"
SET nom = :nom, prenom = :prenom, telephone = :telephone, email = :email,
    "motDePasse" = :motDePasse, role = :role, actif = :actif,
    "doitChangerMotDePasse" = :doitChangerMotDePasse, discipline = :discipline,
    sexe = :sexe, matricule = :matricule, grade = :grade, fonction = :fonction,
    "datePriseService" = :datePriseService, "dateEntreeFonctionPublique" =
:dateEntreeFonctionPublique,
    "acteNomination" = :acteNomination, "majLe" = :majLe
WHERE id = :id;
```

Hibernate réécrit toutes les colonnes de la ligne, même celles qui ne changent pas.

Supprimer

SUPPRIMER R13. Supprimer un compte

Quand : Uniquement quand le Directeur désapprouve la création d'un enseignant, et seulement s'il n'a encore saisi aucune note (voir « Compter les notes saisies par un compte »).

```sql
DELETE FROM "Utilisateur" WHERE id = :id;
```

Ses notifications et les attestations dont il est bénéficiaire partent avec lui ; il n'est plus professeur principal ni professeur des matières (champ remis à vide). La base refuse la suppression s'il est l'auteur de notes, de décisions, de renvois ou d'attestations délivrées.

4. Établissement et barèmes

L'établissement tient sur une seule ligne (id = 1) : ce qui s'imprime en tête des documents, le préfixe des matricules et la moyenne minimale du tableau d'honneur. Les barèmes donnent les grades par matière (GRADE) et les cotes de la moyenne générale (COTE).

Tables : `Etablissement` , `Bareme`

Lire

LIRE R14. Lire l'établissement

Quand : Paramètres ; en-tête de chaque PDF (bulletins, listes, cartes, attestations, certificats) ; préfixe des matricules à l'inscription d'un élève ; seuil du tableau d'honneur.

```
SELECT * FROM "Etablissement" WHERE id = 1;
```

LIRE R15. Savoir si l'établissement existe

Quand : Amorçage d'une base vide.

```
SELECT e.id FROM "Etablissement" e WHERE e.id = 1 LIMIT 1;
```

LIRE R16. Lire les barèmes, triés

Quand : Paramètres (GET /api/baremes).

```
SELECT * FROM "Bareme" ORDER BY type, min DESC;
```

LIRE R17. Lire tous les barèmes

Quand : Calcul des résultats : relevé de notes, bulletins, statistiques, tableau d'honneur.

```
SELECT * FROM "Bareme";
```

LIRE R18. Lire un barème

Quand : Avant de le remplacer : l'ancien est gardé dans le journal des décisions pour pouvoir le rétablir.

```
SELECT * FROM "Bareme" WHERE type = :type;
```

LIRE R19. Compter les barèmes

Quand : Amorçage d'une base vide.

```
SELECT count(*) FROM "Bareme";
```

Ajouter

AJOUTER R20. Créer l'établissement

Quand : Amorçage d'une base vide (une seule fois).

```
INSERT INTO "Etablissement" (id, "nomFr", "nomEn", sigle, "sigleEn", "regionFr", "regionEn",
  "departementFr", "departementEn", arrondissement, localite, code, "boitePostale", telephone,
  "prefixeMatricule", "seuilTableauHonneur")
VALUES (1, :nomFr, :nomEn, :sigle, :sigleEn, :regionFr, :regionEn, :departementFr, :departementEn,
  :arrondissement, :localite, :code, :boitePostale, :telephone, :prefixeMatricule,
:seuilTableauHonneur);
```

AJOUTER R21. Ajouter une mention

Quand : Une fois par mention du nouveau barème ; amorçage (barèmes par défaut).

```
INSERT INTO "Bareme" (id, type, libelle, "libelleLong", min)
VALUES (:id, :type, :libelle, :libelleLong, :min);
```

Modifier

MODIFIER R22. Mettre à jour l'établissement

Quand : Paramètres : informations de l'établissement (PUT /api/etablissement, SG) ou moyenne minimale du tableau d'honneur (PUT /api/etablissement/tableau-honneur, Directeur ou SG) ; désapprobation.

```
UPDATE "Etablissement"
SET "nomFr" = :nomFr, "nomEn" = :nomEn, sigle = :sigle, "sigleEn" = :sigleEn,
    "regionFr" = :regionFr, "regionEn" = :regionEn, "departementFr" = :departementFr,
    "departementEn" = :departementEn, arrondissement = :arrondissement, localite = :localite,
    code = :code, "boitePostale" = :boitePostale, telephone = :telephone,
    "prefixeMatricule" = :prefixeMatricule, "seuilTableauHonneur" = :seuilTableauHonneur
WHERE id = 1;
```

Supprimer

SUPPRIMER R23. Supprimer un barème entier

Quand : Remplacer le barème des grades ou des cotes (PUT /api/baremes/{type}) ; désapprobation qui rétablit l'ancien. Les nouvelles mentions sont ajoutées juste après.

```
DELETE FROM "Bareme" WHERE type = :type;
```

5. Années scolaires et séquences

Une année compte six séquences, deux par trimestre. Une seule année est « en cours » (active) ; c'est elle que l'application utilise quand aucune année n'est choisie.

Tables : `AnneeScolaire` , `Sequence`

Lire

LIRE R24. Lister les années

Quand : Menu « Année scolaire », page Années et séquences (GET /api/annees).

```
SELECT * FROM "AnneeScolaire" ORDER BY libelle DESC;
```

LIRE R25. Compter les classes de chaque année

Quand : Page Années et séquences.

```
SELECT c."anneeId", count(*) FROM "Classe" c GROUP BY c."anneeId";
```

LIRE R26. Compter les inscriptions de chaque année

Quand : Page Années et séquences.

```
SELECT i."anneeId", count(*) FROM "Inscription" i GROUP BY i."anneeId";
```

LIRE R27. Lire l'année en cours

Quand : Toute requête qui ne précise pas d'année ; création d'une année ; attestation (année imprimée).

```
SELECT * FROM "AnneeScolaire" WHERE active = true LIMIT 1;
```

LIRE R28. Chercher une autre année en cours

Quand : Activer une année : l'ancienne année en cours est gardée dans le journal.

```
SELECT * FROM "AnneeScolaire" WHERE active = true AND id <> :id LIMIT 1;
```

LIRE R29. Lire une année par son identifiant

Quand : Année choisie dans le menu, activation, création de classes et de groupes, documents, tableau d'honneur.

```
SELECT * FROM "AnneeScolaire" WHERE id = :id;
```

LIRE R30. Compter les années

Quand : Amorçage d'une base vide.

```
SELECT count(*) FROM "AnneeScolaire";
```

LIRE R31. Lire les séquences d'une année

Quand : Avancement de la saisie (tableau de bord).

```
SELECT * FROM "Sequence" WHERE "anneeId" = :anneeId ORDER BY numero;
```

LIRE R32. Lire les séquences de plusieurs années

Quand : Page Années et séquences.

```
SELECT * FROM "Sequence" WHERE "anneeId" IN (:anneeIds) ORDER BY numero;
```

LIRE R33. Lire une séquence

Quand : Feuille de notes, élèves sans note, ouverture ou fermeture de la saisie.

```
SELECT * FROM "Sequence" WHERE id = :id;
```

LIRE R34. Compter les inscriptions d'une année

Quand : Avant de supprimer une année : une année où des élèves sont inscrits ne peut pas être retirée.

```
SELECT count(*) FROM "Inscription" WHERE "anneeId" = :anneeId;
```

Ajouter

AJOUTER R35. Créer une année

Quand : Nouvelle année (POST /api/annees) ; amorçage.

```
INSERT INTO "AnneeScolaire" (id, libelle, active, "creeLe")
VALUES (:id, :libelle, :active, :creeLe);
```

AJOUTER R36. Créer une séquence

Quand : Six fois à la création d'une année (numéros 1 à 6, trimestres 1 à 3).

```
INSERT INTO "Sequence" (id, "anneeId", numero, trimestre, "saisieOuverte")
VALUES (:id, :anneeId, :numero, :trimestre, false);
```

Modifier

MODIFIER R37. Mettre à jour une année

Quand : Rendre une année active (POST /api/annees/{id}/activer) ; désapprobation.

```
UPDATE "AnneeScolaire" SET active = :active, libelle = :libelle WHERE id = :id;
```

MODIFIER R38. Désactiver les autres années

Quand : Juste après l'activation d'une année : une seule reste en cours.

```
UPDATE "AnneeScolaire" SET active = false WHERE id <> :id;
```

MODIFIER R39. Ouvrir ou fermer la saisie

Quand : Années et séquences (PATCH /api/annees/sequences/{id}) ; désapprobation.

```
UPDATE "Sequence"
SET "anneeId" = :anneeId, numero = :numero, trimestre = :trimestre, "saisieOuverte" = :saisieOuverte
WHERE id = :id;
```

Supprimer

SUPPRIMER R40. Supprimer une année

Quand : Uniquement quand le Directeur désapprouve la création d'une année, si aucun élève n'y est inscrit.

```sql
DELETE FROM "AnneeScolaire" WHERE id = :id;
```

Ses séquences, classes (avec leurs matières), groupes de matières et inscriptions sont supprimés avec elle par la base.

6. Spécialités et classes

Les spécialités (Mécanique, Électricité…) servent d'une année sur l'autre. Une classe appartient à une année ; son nom se calcule à partir du niveau, de la spécialité et d'une lettre A, B… quand le niveau et la spécialité comptent plusieurs classes (colonne numero).

Tables : `Specialite` , `Classe`

Lire

LIRE R41. Lister les spécialités

Quand : Fenêtre Spécialités, formulaire de classe (GET /api/specialites).

```
SELECT * FROM "Specialite" ORDER BY nom;
```

LIRE R42. Compter les classes de chaque spécialité

Quand : Fenêtre Spécialités (une spécialité utilisée ne peut pas être supprimée).

```
SELECT c."specialiteId", count(*) FROM "Classe" c
WHERE c."specialiteId" IS NOT NULL
GROUP BY c."specialiteId";
```

LIRE R43. Compter les classes d'une spécialité

Quand : Avant de modifier ou de supprimer une spécialité.

```
SELECT count(*) FROM "Classe" WHERE "specialiteId" = :specialiteId;
```

LIRE R44. Lire une spécialité

Quand : Création ou modification d'une classe, d'une spécialité ; désapprobation.

```
SELECT * FROM "Specialite" WHERE id = :id;
```

LIRE R45. Années et niveaux des classes d'une spécialité

Quand : Renommer une spécialité : les classes qui la portent sont renommées, groupe par groupe.

```
SELECT DISTINCT c."anneeId", c.niveau FROM "Classe" c WHERE c."specialiteId" = :specialiteId;
```

LIRE R46. Lister les classes de l'année

Quand : Page Classes et tous les choix de classe (GET /api/classes) ; copie de l'organisation vers une nouvelle année.

```
SELECT c.*, s.*, u.*
FROM "Classe" c
LEFT JOIN "Specialite" s ON s.id = c."specialiteId"
LEFT JOIN "Utilisateur" u ON u.id = c."profPrincipalId"
WHERE c."anneeId" = :anneeId
ORDER BY c.niveau, s.nom, c.numero;
```

LIRE R47. Lister les classes d'un professeur

Quand : Un professeur ne voit que les classes où il enseigne (GET /api/classes).

```
SELECT c.*, s.*, u.*
FROM "Classe" c
LEFT JOIN "Specialite" s ON s.id = c."specialiteId"
LEFT JOIN "Utilisateur" u ON u.id = c."profPrincipalId"
WHERE c."anneeId" = :anneeId
  AND EXISTS (SELECT 1 FROM "Matiere" m WHERE m."classeId" = c.id AND m."professeurId" =
:professeurId)
ORDER BY c.niveau, s.nom, c.numero;
```

LIRE R48. Lister les classes de l'année par niveau et nom

Quand : Statistiques de l'établissement, tableau d'honneur.

```
SELECT * FROM "Classe" WHERE "anneeId" = :anneeId ORDER BY niveau, nom;
```

LIRE R49. Lire des classes choisies

Quand : Bulletins de classes entières (POST /api/documents/bulletins).

```
SELECT * FROM "Classe" WHERE id IN (:ids) ORDER BY niveau, nom;
```

LIRE R50. Lire plusieurs classes

Quand : Création ou modification d'un groupe (ses classes) ; désapprobation.

```
SELECT * FROM "Classe" WHERE id IN (:ids);
```

LIRE R51. Lire une classe

Quand : Fiche, modification ou suppression d'une classe ; liste et relevé d'une classe ; résultats ; inscription d'un élève.

```
SELECT * FROM "Classe" WHERE id = :id;
```

LIRE R52. Vérifier qu'une classe est de l'année

Quand : Création d'une matière.

```
SELECT count(*) FROM "Classe" WHERE id = :id AND "anneeId" = :anneeId;
```

LIRE R53. Classes qui partagent les lettres A, B...

Quand : Création, modification ou suppression d'une classe : les classes du même niveau et de la même spécialité sont renommées.

```
SELECT * FROM "Classe" c
WHERE c."anneeId" = :anneeId AND c.niveau = :niveau
  AND ((:specialiteId IS NULL AND c."specialiteId" IS NULL) OR c."specialiteId" = :specialiteId)
ORDER BY c.numero, c.nom;
```

LIRE R54. Plus grand numéro du groupe

Quand : Nouvelle classe : elle prend le numéro suivant (lettre suivante).

```
SELECT coalesce(max(c.numero), 0) FROM "Classe" c
WHERE c."anneeId" = :anneeId AND c.niveau = :niveau
  AND ((:specialiteId IS NULL AND c."specialiteId" IS NULL) OR c."specialiteId" = :specialiteId);
```

LIRE R55. Classes dont des professeurs sont principaux

Quand : Page Enseignants.

```
SELECT * FROM "Classe" c WHERE c."anneeId" = :anneeId AND c."profPrincipalId" IN (:professeurIds);
```

LIRE R56. Compter les inscriptions d'une classe

Quand : Avant de supprimer une classe : une classe avec des élèves ne peut pas être supprimée.

```
SELECT count(*) FROM "Inscription" WHERE "classeId" = :classeId;
```

Ajouter

AJOUTER R57. Créer une spécialité

Quand : Fenêtre Spécialités (POST /api/specialites) ; désapprobation d'une suppression.

```
INSERT INTO "Specialite" (id, nom, "nomEn", "creeLe") VALUES (:id, :nom, :nomEn, :creeLe);
```

AJOUTER R58. Créer une classe

Quand : Nouvelle classe (POST /api/classes) ; nouvelle année avec reprise de l'organisation ; désapprobation d'une suppression.

```
INSERT INTO "Classe" (id, "anneeId", nom, "nomEn", niveau, numero, "specialiteId", "profPrincipalId")
VALUES (:id, :anneeId, :nom, :nomEn, :niveau, :numero, :specialiteId, :profPrincipalId);
```

Modifier

MODIFIER R59. Renommer une spécialité

Quand : PUT /api/specialites/{id} ; désapprobation.

```
UPDATE "Specialite" SET nom = :nom, "nomEn" = :nomEn WHERE id = :id;
```

MODIFIER R60. Mettre à jour une classe

Quand : Modifier une classe (PUT /api/classes/{id}) ; renommage automatique des classes voisines (lettres) ; désapprobation.

```
UPDATE "Classe"
SET "anneeId" = :anneeId, nom = :nom, "nomEn" = :nomEn, niveau = :niveau, numero = :numero,
    "specialiteId" = :specialiteId, "profPrincipalId" = :profPrincipalId
WHERE id = :id;
```

Supprimer

SUPPRIMER R61. Supprimer une spécialité

Quand : DELETE /api/specialites/{id}, si aucune classe ne la porte ; désapprobation d'une création.

```
DELETE FROM "Specialite" WHERE id = :id;
```

La base refuse la suppression tant qu'une classe porte la spécialité.

SUPPRIMER R62. Supprimer une classe

Quand : DELETE /api/classes/{id}, si aucun élève n'y est inscrit ; désapprobation d'une création. Ses matières sont d'abord supprimées une à une (voir Matières).

```
DELETE FROM "Classe" WHERE id = :id;
```

Ses rattachements aux groupes de matières disparaissent avec elle. La base refuse la suppression si une inscription la référence.

7. Groupes de matières

Un groupe (matières générales, professionnelles…) regroupe les matières sur le bulletin. Il est rattaché aux classes qui l'utilisent par la table de liaison _ClasseToGroupeMatiere (colonne A : la classe, colonne B : le groupe).

Tables : `GroupeMatiere` , `_ClasseToGroupeMatiere`

Lire

LIRE R63. Lister les groupes de l'année

Quand : Page Groupes de matières (GET /api/groupes) ; copie vers une nouvelle année.

```
SELECT * FROM "GroupeMatiere" WHERE "anneeId" = :anneeId ORDER BY ordre;
```

LIRE R64. Classes rattachées à des groupes

Quand : Page Groupes de matières.

```
SELECT cg."B" AS "groupeId", c.*
FROM "_ClasseToGroupeMatiere" cg
JOIN "Classe" c ON c.id = cg."A"
WHERE cg."B" IN (:groupeIds);
```

LIRE R65. Groupes rattachés à des classes

Quand : Page Classes ; fiche d'une classe ; copie vers une nouvelle année.

```
SELECT cg."A" AS "classeId", g.*
FROM "_ClasseToGroupeMatiere" cg
JOIN "GroupeMatiere" g ON g.id = cg."B"
WHERE cg."A" IN (:classeIds);
```

LIRE R66. Compter les matières de chaque groupe

Quand : Page Groupes de matières.

```
SELECT m."groupeId", count(*) FROM "Matiere" m WHERE m."groupeId" IN (:groupeIds) GROUP BY
m."groupeId";
```

LIRE R67. Lire un groupe

Quand : Modification ou suppression d'un groupe ; feuille de notes ; désapprobation.

```
SELECT * FROM "GroupeMatiere" WHERE id = :id;
```

LIRE R68. Lire plusieurs groupes

Quand : Désapprobation de la suppression d'une classe : ses rattachements sont rétablis.

```
SELECT * FROM "GroupeMatiere" WHERE id IN (:ids);
```

LIRE R69. Le groupe est-il rattaché à la classe ?

Quand : Création ou modification d'une matière : son groupe doit être rattaché à sa classe.

```
SELECT g.id
FROM "GroupeMatiere" g
JOIN "_ClasseToGroupeMatiere" cg ON cg."B" = g.id
WHERE g.id = :groupeId AND cg."A" = :classeId
LIMIT 1;
```

LIRE R70. Compter les matières du groupe

Quand : Avant de supprimer un groupe (il doit être vide) ; désapprobation.

```
SELECT count(*) FROM "Matiere" WHERE "groupeId" = :groupeId;
```

LIRE R71. Matières du groupe hors des classes gardées

Quand : Modifier les classes d'un groupe : on ne peut pas retirer une classe dont des matières sont dans le groupe.

```
SELECT count(*) FROM "Matiere" m WHERE m."groupeId" = :groupeId AND m."classeId" NOT IN (:classeIds);
```

LIRE R72. Noms des classes qui bloquent

Quand : Même vérification : le message d'erreur cite ces classes.

```
SELECT DISTINCT c.nom
FROM "Matiere" m
JOIN "Classe" c ON c.id = m."classeId"
WHERE m."groupeId" = :groupeId AND m."classeId" NOT IN (:classeIds);
```

LIRE R73. Matières du groupe dans des classes données

Quand : Désapprobation de la modification d'un groupe.

```
SELECT count(*) FROM "Matiere" m WHERE m."groupeId" = :groupeId AND m."classeId" IN (:classeIds);
```

Ajouter

AJOUTER R74. Créer un groupe

Quand : Nouveau groupe (POST /api/groupes) ; nouvelle année avec reprise de l'organisation ; désapprobation d'une suppression.

```
INSERT INTO "GroupeMatiere" (id, "anneeId", nom, "nomEn", ordre)
VALUES (:id, :anneeId, :nom, :nomEn, :ordre);
```

AJOUTER R75. Rattacher une classe au groupe

Quand : Une fois par classe, à la création ou à la modification du groupe.

```
INSERT INTO "_ClasseToGroupeMatiere" ("A", "B") VALUES (:classeId, :groupeId);
```

Modifier

MODIFIER R76. Mettre à jour un groupe

Quand : PUT /api/groupes/{id} ; désapprobation.

```
UPDATE "GroupeMatiere" SET "anneeId" = :anneeId, nom = :nom, "nomEn" = :nomEn, ordre = :ordre WHERE id
= :id;
```

Supprimer

SUPPRIMER R77. Détacher une classe d'un groupe

Quand : Modification du groupe (classe retirée) ; désapprobation.

```
DELETE FROM "_ClasseToGroupeMatiere" WHERE "B" = :groupeId AND "A" = :classeId;
```

SUPPRIMER R78. Détacher toutes les classes du groupe

Quand : Juste avant la suppression du groupe.

```
DELETE FROM "_ClasseToGroupeMatiere" WHERE "B" = :groupeId;
```

SUPPRIMER R79. Supprimer un groupe

Quand : DELETE /api/groupes/{id}, s'il ne contient plus de matière ; désapprobation d'une création.

```
DELETE FROM "GroupeMatiere" WHERE id = :id;
```

La base refuse la suppression tant qu'une matière est rangée dans le groupe.

8. Matières

Une matière appartient à une classe et à un groupe rattaché à cette classe ; elle a un coefficient, un ordre sur le bulletin et au plus un professeur.

Tables : `Matiere`

Lire

LIRE R80. Lister les matières de l'année

Quand : Page Matières (filtre par classe), matières d'un professeur (saisie des notes, statistiques de ses matières) : GET /api/matieres.

```
SELECT m.*, c.*, g.*, p.*
FROM "Matiere" m
JOIN "Classe" c ON c.id = m."classeId"
JOIN "GroupeMatiere" g ON g.id = m."groupeId"
LEFT JOIN "Utilisateur" p ON p.id = m."professeurId"
WHERE c."anneeId" = :anneeId
  AND (:classeId IS NULL OR c.id = :classeId)
  AND (:professeurId IS NULL OR m."professeurId" = :professeurId)
ORDER BY c.niveau, g.ordre, m.ordre;
```

LIRE R81. Matières suivies pour la saisie

Quand : Tableau de bord (avancement de la saisie) ; élèves sans note.

```
SELECT m.*, c.*, p.*
FROM "Matiere" m
JOIN "Classe" c ON c.id = m."classeId"
LEFT JOIN "Utilisateur" p ON p.id = m."professeurId"
WHERE c."anneeId" = :anneeId
  AND (:matiereId IS NULL OR m.id = :matiereId)
  AND (:classeId IS NULL OR c.id = :classeId)
  AND (:professeurId IS NULL OR m."professeurId" = :professeurId)
ORDER BY c.niveau, m.ordre;
```

LIRE R82. Matières d'une classe pour le calcul

Quand : Relevé de notes, bulletins, statistiques, tableau d'honneur : tout ce qu'il faut pour calculer les résultats d'une classe.

```
SELECT m.*, g.*, p.*
FROM "Matiere" m
JOIN "GroupeMatiere" g ON g.id = m."groupeId"
LEFT JOIN "Utilisateur" p ON p.id = m."professeurId"
WHERE m."classeId" = :classeId;
```

LIRE R83. Matières d'une classe

Quand : Avant de supprimer une classe (ses matières partent avec elle) ; désapprobation.

```
SELECT * FROM "Matiere" WHERE "classeId" = :classeId;
```

LIRE R84. Matières de plusieurs classes

Quand : Nouvelle année avec reprise de l'organisation.

```
SELECT * FROM "Matiere" WHERE "classeId" IN (:classeIds);
```

LIRE R85. Matières enseignées par des professeurs

Quand : Page Enseignants (matières et classes de chacun).

```
SELECT m.*, c.*
FROM "Matiere" m
JOIN "Classe" c ON c.id = m."classeId"
WHERE c."anneeId" = :anneeId AND m."professeurId" IN (:professeurIds)
ORDER BY c.niveau;
```

LIRE R86. Compter les matières de chaque classe

Quand : Page Classes.

```
SELECT m."classeId", count(*) FROM "Matiere" m WHERE m."classeId" IN (:classeIds) GROUP BY
m."classeId";
```

LIRE R87. Lire une matière

Quand : Modification ou suppression ; feuille de notes (le professeur doit être celui de la matière).

```
SELECT * FROM "Matiere" WHERE id = :id;
```

LIRE R88. Compter les notes d'une matière

Quand : Avant de modifier ou de supprimer une matière : une matière notée demande une confirmation et ne change plus de classe.

```
SELECT count(*) FROM "Note" WHERE "matiereId" = :matiereId;
```

Ajouter

AJOUTER R89. Créer une matière

Quand : Nouvelle matière (POST /api/matieres) ; nouvelle année avec reprise de l'organisation ; désapprobation d'une suppression.

```
INSERT INTO "Matiere" (id, "classeId", "groupeId", nom, "nomEn", coefficient, ordre, "professeurId")
VALUES (:id, :classeId, :groupeId, :nom, :nomEn, :coefficient, :ordre, :professeurId);
```

Modifier

MODIFIER R90. Mettre à jour une matière

Quand : PUT /api/matieres/{id} ; désapprobation.

```
UPDATE "Matiere"
SET "classeId" = :classeId, "groupeId" = :groupeId, nom = :nom, "nomEn" = :nomEn,
    coefficient = :coefficient, ordre = :ordre, "professeurId" = :professeurId
WHERE id = :id;
```

Supprimer

SUPPRIMER R91. Supprimer une matière

Quand : DELETE /api/matieres/{id} (avec confirmation si elle a des notes) ; suppression d'une classe (une fois par matière) ; désapprobation d'une création.

```
DELETE FROM "Matiere" WHERE id = :id;
```

Ses notes et ses compétences visées sont supprimées avec elle par la base.

9. Élèves, inscriptions et renvois

Un élève a une fiche (Eleve) et une inscription par année (Inscription), dans une classe. Les notes et les renvois portent sur l'inscription : c'est ce qui permet de réinscrire un élève l'année suivante sans perdre son historique. Le matricule est attribué par un compteur.

Tables : `Eleve` , `Inscription` , `Renvoi` , `Compteur`

Lire

LIRE R92. Lister les élèves de l'année

Quand : Page Élèves, élèves renvoyés, réinscriptions (GET /api/eleves). Les conditions entre crochets ne sont ajoutées que si le filtre est donné : statut (ACTIF par défaut, RENVOYE ou TOUS), classe, recherche.

```
SELECT i.*, e.*, c.*, a.*
FROM "Inscription" i
JOIN "Eleve" e ON e.id = i."eleveId"
JOIN "Classe" c ON c.id = i."classeId"
JOIN "AnneeScolaire" a ON a.id = i."anneeId"
WHERE i."anneeId" = :anneeId
  AND i.statut = :statut                    -- [statut]
  AND c.id = :classeId                      -- [classe]
  AND (lower(e.nom) LIKE :motif ESCAPE '\'    -- [recherche : « %texte% »]
    OR lower(e.prenoms) LIKE :motif ESCAPE '\'
    OR lower(e.matricule) LIKE :motif ESCAPE '\'
    OR lower(e."matriculeNational") LIKE :motif ESCAPE '\')
ORDER BY c.niveau, e.nom, e.prenoms;
```

LIRE R93. Renvois des élèves listés

Quand : Même page, et fiche de l'élève : le dernier renvoi de chacun, avec son auteur.

```
SELECT r.*, p.*, rp.*
FROM "Renvoi" r
JOIN "Utilisateur" p ON p.id = r."parId"
LEFT JOIN "Utilisateur" rp ON rp.id = r."reintegreParId"
WHERE r."inscriptionId" IN (:inscriptionIds)
ORDER BY r."creeLe" DESC;
```

LIRE R94. Élèves actifs d'une classe

Quand : Feuille de notes, liste de classe (PDF, Excel), relevé, calcul des résultats.

```
SELECT i.*, e.*
FROM "Inscription" i
JOIN "Eleve" e ON e.id = i."eleveId"
WHERE i."classeId" = :classeId AND i.statut = 'ACTIF'
ORDER BY e.nom, e.prenoms;
```

LIRE R95. Élèves sans note

Quand : Élèves sans note d'une matière pour une séquence (GET /api/notes/manquantes).

```
SELECT i.*, e.*
FROM "Inscription" i
JOIN "Eleve" e ON e.id = i."eleveId"
WHERE i."classeId" = :classeId AND i.statut = 'ACTIF'
  AND NOT EXISTS (SELECT 1 FROM "Note" n
                  WHERE n."inscriptionId" = i.id AND n."matiereId" = :matiereId AND n."sequenceId" =
:sequenceId)
ORDER BY e.nom, e.prenoms;
```

LIRE R96. Inscriptions encore actives dans la classe

Quand : Enregistrement d'une feuille de notes : chaque note doit viser un élève actif de la classe.

```
SELECT i.id FROM "Inscription" i
WHERE i.id IN (:ids) AND i."classeId" = :classeId AND i.statut = 'ACTIF';
```

LIRE R97. Inscriptions actives choisies

Quand : Bulletins d'élèves choisis un à un.

```
SELECT * FROM "Inscription" i WHERE i.id IN (:ids) AND i.statut = 'ACTIF';
```

LIRE R98. Élèves pour les cartes scolaires

Quand : Cartes scolaires et vérification des photos : classes entières et élèves choisis.

```
SELECT i.*, e.*, c.*, a.*
FROM "Inscription" i
JOIN "Eleve" e ON e.id = i."eleveId"
JOIN "Classe" c ON c.id = i."classeId"
JOIN "AnneeScolaire" a ON a.id = i."anneeId"
WHERE i.statut = 'ACTIF' AND (c.id IN (:classeIds) OR i.id IN (:inscriptionIds))
ORDER BY c.niveau, c.nom, e.nom, e.prenoms;
```

LIRE R99. Élèves déjà inscrits dans l'année

Quand : Réinscription : un élève ne peut pas être inscrit deux fois la même année.

```
SELECT i."eleveId" FROM "Inscription" i WHERE i."anneeId" = :anneeId AND i."eleveId" IN (:eleveIds);
```

LIRE R100. Parcours d'un élève

Quand : Fiche de l'élève : ses inscriptions, de la plus récente à la plus ancienne.

```
SELECT i.*, c.*, a.*
FROM "Inscription" i
JOIN "Classe" c ON c.id = i."classeId"
JOIN "AnneeScolaire" a ON a.id = i."anneeId"
WHERE i."eleveId" = :eleveId
ORDER BY a.libelle DESC;
```

LIRE R101. Inscription d'un élève pour une année

Quand : Modification de la fiche (changement de classe).

```
SELECT * FROM "Inscription" WHERE "eleveId" = :eleveId AND "anneeId" = :anneeId;
```

LIRE R102. Lire une inscription

Quand : Renvoyer, réintégrer ; désapprobation.

```
SELECT * FROM "Inscription" WHERE id = :id;
```

LIRE R103. Effectif actif de chaque classe

Quand : Page Classes.

```
SELECT i."classeId", count(*) FROM "Inscription" i
WHERE i."classeId" IN (:classeIds) AND i.statut = 'ACTIF'
GROUP BY i."classeId";
```

LIRE R104. Effectif actif des classes de l'année

Quand : Tableau de bord (avancement de la saisie).

```
SELECT i."classeId", count(*) FROM "Inscription" i
WHERE i."anneeId" = :anneeId AND i.statut = 'ACTIF'
GROUP BY i."classeId";
```

LIRE R105. Effectif actif d'une classe

Quand : Élèves sans note.

```
SELECT count(*) FROM "Inscription" WHERE "classeId" = :classeId AND statut = :statut;
```

LIRE R106. Élèves de l'année par statut

Quand : Statistiques de l'établissement : effectif, élèves renvoyés.

```
SELECT i.statut, count(*) FROM "Inscription" i WHERE i."anneeId" = :anneeId GROUP BY i.statut;
```

LIRE R107. Lire un élève

Quand : Fiche de l'élève, photo, modification ; désapprobation.

```
SELECT * FROM "Eleve" WHERE id = :id;
```

LIRE R108. Lire plusieurs élèves

Quand : Photos de plusieurs élèves ; réinscription.

```
SELECT * FROM "Eleve" WHERE id IN (:ids);
```

LIRE R109. Élèves qui ont une photo

Quand : Désapprobation d'un ajout d'élèves : leurs photos sont aussi effacées du stockage.

```
SELECT * FROM "Eleve" WHERE id IN (:ids) AND "photoCle" IS NOT NULL;
```

LIRE R110. Renvoi en cours

Quand : Réintégrer : le renvoi le plus récent qui n'est pas encore levé.

```
SELECT * FROM "Renvoi"
WHERE "inscriptionId" = :inscriptionId AND "reintegreLe" IS NULL
ORDER BY "creeLe" DESC;
```

LIRE R111. Compter les notes d'une inscription

Quand : Avant de changer un élève de classe (impossible s'il a des notes) ; désapprobation.

```
SELECT count(*) FROM "Note" WHERE "inscriptionId" = :inscriptionId;
```

LIRE R112. Compter les notes de plusieurs inscriptions

Quand : Désapprobation d'une réinscription : refusée si des notes ont été saisies depuis.

```
SELECT count(*) FROM "Note" WHERE "inscriptionId" IN (:inscriptionIds);
```

LIRE R113. Compter les notes d'un ou de plusieurs élèves

Quand : Désapprobation d'une inscription ou d'un ajout groupé : refusée si des notes ont été saisies depuis.

```
SELECT count(*)
FROM "Note" n
JOIN "Inscription" i ON i.id = n."inscriptionId"
WHERE i."eleveId" IN (:eleveIds);
```

Ajouter

AJOUTER R114. Attribuer le numéro suivant

Quand : Nouveau matricule d'élève (clé « matricule-CTM-26 ») ; numéro d'attestation (clé « attestation-2026-2027 »). Une seule instruction : deux inscriptions simultanées ne reçoivent jamais le même numéro.

```
INSERT INTO "Compteur" (cle, valeur) VALUES (:cle, 1)
ON CONFLICT (cle) DO UPDATE SET valeur = "Compteur".valeur + 1
RETURNING valeur;
```

AJOUTER R115. Créer la fiche d'un élève

Quand : Inscrire un élève (POST /api/eleves), ajout groupé (POST /api/eleves/lot), import Excel (POST /api/eleves/import).

```
INSERT INTO "Eleve" (id, matricule, "matriculeNational", nom, prenoms, sexe, "dateNaissance",
  "lieuNaissance", "photoCle", "parentNom", "parentTelephone", "parentAdresse", "creeLe", "majLe")
VALUES (:id, :matricule, :matriculeNational, :nom, :prenoms, :sexe, :dateNaissance,
  :lieuNaissance, :photoCle, :parentNom, :parentTelephone, :parentAdresse, :creeLe, :majLe);
```

AJOUTER R116. Inscrire un élève dans une classe

Quand : Mêmes actions, et réinscription dans une nouvelle année (POST /api/eleves/reinscrire).

```
INSERT INTO "Inscription" (id, "eleveId", "anneeId", "classeId", redoublant, statut, "creeLe")
VALUES (:id, :eleveId, :anneeId, :classeId, :redoublant, 'ACTIF', :creeLe);
```

AJOUTER R117. Enregistrer un renvoi

Quand : Renvoyer un élève (POST /api/eleves/inscriptions/{id}/renvoyer, Directeur).

```
INSERT INTO "Renvoi" (id, "inscriptionId", date, motif, "parId", "creeLe", "reintegreLe",
"reintegreParId", "motifReintegration")
VALUES (:id, :inscriptionId, :date, :motif, :parId, :creeLe, NULL, NULL, NULL);
```

Modifier

MODIFIER R118. Mettre à jour la fiche d'un élève

Quand : Modifier la fiche (PUT /api/eleves/{id}) ; ajouter ou changer sa photo (une ou plusieurs) ; désapprobation.

```
UPDATE "Eleve"
SET matricule = :matricule, "matriculeNational" = :matriculeNational, nom = :nom, prenoms = :prenoms,
    sexe = :sexe, "dateNaissance" = :dateNaissance, "lieuNaissance" = :lieuNaissance,
    "photoCle" = :photoCle, "parentNom" = :parentNom, "parentTelephone" = :parentTelephone,
    "parentAdresse" = :parentAdresse, "majLe" = :majLe
WHERE id = :id;
```

MODIFIER R119. Mettre à jour une inscription

Quand : Changer l'élève de classe ou de statut redoublant ; renvoi (statut RENVOYE) ; réintégration (statut ACTIF) ; désapprobation.

```
UPDATE "Inscription"
SET "eleveId" = :eleveId, "anneeId" = :anneeId, "classeId" = :classeId,
    redoublant = :redoublant, statut = :statut
WHERE id = :id;
```

MODIFIER R120. Lever un renvoi

Quand : Réintégrer un élève (POST /api/eleves/inscriptions/{id}/reintegrer) : date, auteur et observation de la réintégration.

```
UPDATE "Renvoi"
SET "inscriptionId" = :inscriptionId, date = :date, motif = :motif, "parId" = :parId,
    "reintegreLe" = :reintegreLe, "reintegreParId" = :reintegreParId,
    "motifReintegration" = :motifReintegration
WHERE id = :id;
```

Supprimer

SUPPRIMER R121. Supprimer des inscriptions

Quand : Uniquement quand le Directeur désapprouve une réinscription.

```
DELETE FROM "Inscription" WHERE id IN (:ids);
```

Les notes et les renvois de ces inscriptions sont supprimés avec elles par la base.

SUPPRIMER R122. Supprimer des élèves

Quand : Uniquement quand le Directeur désapprouve une inscription, un ajout groupé ou un import, si aucune note n'a été saisie depuis.

```
DELETE FROM "Eleve" WHERE id IN (:ids);
```

Leurs inscriptions, et avec elles leurs notes et leurs renvois, sont supprimées par la base.

10. Notes et compétences visées

Une note relie une inscription, une matière et une séquence (une seule note par triplet). La compétence visée est le texte que le professeur associe à une matière pour une séquence ; elle s'imprime sur le bulletin.

Tables : `Note` , `CompetenceVisee`

Lire

LIRE R123. Notes d'une feuille

Quand : Feuille de notes d'une matière pour une séquence (GET /api/notes/feuille).

```
SELECT * FROM "Note" WHERE "matiereId" = :matiereId AND "sequenceId" = :sequenceId;
```

LIRE R124. Lire une note

Quand : Enregistrement d'une feuille : la note existe-t-elle déjà ?

```
SELECT * FROM "Note"
WHERE "inscriptionId" = :inscriptionId AND "matiereId" = :matiereId AND "sequenceId" = :sequenceId;
```

LIRE R125. Notes d'une classe pour le calcul

Quand : Relevé de notes, bulletins, statistiques, classement, tableau d'honneur : les notes des élèves actifs pour les séquences de la période (une séquence, les deux d'un trimestre ou les six de l'année).

```
SELECT n."inscriptionId", n."matiereId", s.numero, n.valeur
FROM "Note" n
JOIN "Sequence" s ON s.id = n."sequenceId"
JOIN "Inscription" i ON i.id = n."inscriptionId"
JOIN "Matiere" m ON m.id = n."matiereId"
WHERE m."classeId" = :classeId AND i.statut = 'ACTIF' AND s.numero IN (:numeros);
```

LIRE R126. Compter les notes de chaque matière

Quand : Page Matières (une matière notée affiche son nombre de notes).

```
SELECT n."matiereId", count(*) FROM "Note" n WHERE n."matiereId" IN (:matiereIds) GROUP BY
n."matiereId";
```

LIRE R127. Compter les notes par matière et par séquence

Quand : Tableau de bord : avancement de la saisie (élèves actifs seulement).

```
SELECT n."matiereId", n."sequenceId", count(*)
FROM "Note" n
JOIN "Inscription" i ON i.id = n."inscriptionId"
WHERE n."matiereId" IN (:matiereIds) AND i.statut = 'ACTIF'
GROUP BY n."matiereId", n."sequenceId";
```

LIRE R128. Compter les notes saisies par un compte

Quand : Désapprobation de la création d'un enseignant : refusée s'il a déjà saisi des notes.

```
SELECT count(*) FROM "Note" WHERE "saisieParId" = :utilisateurId;
```

LIRE R129. Lire une compétence visée

Quand : Feuille de notes ; avant de l'enregistrer (Hibernate vérifie si elle existe).

```
SELECT * FROM "CompetenceVisee" WHERE "matiereId" = :matiereId AND "sequenceId" = :sequenceId;
```

LIRE R130. Compétences visées d'une classe

Quand : Bulletins : les compétences des matières de la classe pour la séquence.

```
SELECT * FROM "CompetenceVisee" cv
WHERE cv."matiereId" IN (SELECT m.id FROM "Matiere" m WHERE m."classeId" = :classeId)
  AND cv."sequenceId" IN (SELECT s.id FROM "Sequence" s WHERE s."anneeId" = :anneeId AND s.numero =
:numero);
```

Ajouter

AJOUTER R131. Ajouter une note

Quand : Enregistrement d'une feuille de notes (PUT /api/notes/feuille), pour chaque élève qui n'avait pas encore de note.

```
INSERT INTO "Note" (id, "inscriptionId", "matiereId", "sequenceId", valeur, "saisieParId", "majLe")
VALUES (:id, :inscriptionId, :matiereId, :sequenceId, :valeur, :saisieParId, :majLe);
```

AJOUTER R132. Ajouter une compétence visée

Quand : Enregistrement d'une feuille avec une compétence, s'il n'y en avait pas.

```
INSERT INTO "CompetenceVisee" ("matiereId", "sequenceId", texte) VALUES (:matiereId, :sequenceId,
:texte);
```

Modifier

MODIFIER R133. Changer une note

Quand : Même enregistrement, pour un élève qui avait déjà une note : nouvelle valeur et nouvel auteur.

```
UPDATE "Note"
SET "inscriptionId" = :inscriptionId, "matiereId" = :matiereId, "sequenceId" = :sequenceId,
    valeur = :valeur, "saisieParId" = :saisieParId, "majLe" = :majLe
WHERE id = :id;
```

MODIFIER R134. Changer une compétence visée

Quand : Même enregistrement, s'il y en avait déjà une.

```
UPDATE "CompetenceVisee" SET texte = :texte WHERE "matiereId" = :matiereId AND "sequenceId" =
:sequenceId;
```

Supprimer

SUPPRIMER R135. Effacer une note

Quand : Même enregistrement, quand le professeur vide la case d'un élève.

```
DELETE FROM "Note"
WHERE "inscriptionId" = :inscriptionId AND "matiereId" = :matiereId AND "sequenceId" = :sequenceId;
```

SUPPRIMER R136. Effacer une compétence visée

Quand : Même enregistrement, quand le professeur vide la compétence.

```
DELETE FROM "CompetenceVisee" WHERE "matiereId" = :matiereId AND "sequenceId" = :sequenceId;
```

11. Journal des décisions

Chaque action du Surveillant Général sur l'organisation, la scolarité ou l'administration est consignée ici, approuvée d'office. La colonne annulation (JSON) garde ce qu'il faut pour défaire l'action si le Directeur la désapprouve dans les 4 semaines.

Tables : `Decision`

Lire

LIRE R137. Lire une page du journal

Quand : Décisions prises (GET /api/decisions), 50 par page, les plus récentes d'abord. Les conditions entre crochets ne sont ajoutées que si le filtre est donné ; la dernière sert à charger la page suivante.

```sql
SELECT * FROM "Decision" d
WHERE d.domaine = :domaine                                  -- [domaine]
  AND d.statut = :statut                                    -- [statut]
  AND d."creeLe" >= :debutDelai                             -- [statut « encore désapprouvables »]
  AND lower(d.libelle) LIKE :motif ESCAPE '\'               -- [recherche]
  AND (d."creeLe" < :curseurDate                            -- [page suivante]
       OR (d."creeLe" = :curseurDate AND d.id < :curseurId))
ORDER BY d."creeLe" DESC, d.id DESC
LIMIT 51;
```

LIRE R138. Compter les décisions encore désapprouvables

Quand : En tête du journal.

```sql
SELECT count(*) FROM "Decision" WHERE statut = 'APPROUVEE' AND "creeLe" >= :debutDelai;
```

LIRE R139. Compter les décisions désapprouvées

Quand : En tête du journal.

```sql
SELECT count(*) FROM "Decision" WHERE statut = 'DESAPPROUVEE';
```

LIRE R140. Lire une décision

Quand : Désapprouver ; curseur de la page suivante.

```sql
SELECT * FROM "Decision" WHERE id = :id;
```

Ajouter

AJOUTER R141. Consigner une décision

Quand : Après chaque action du SG concernée : classe, spécialité, groupe, matière, enseignant, élève (inscription, fiche, photo, import, réinscription), année, séquence, établissement, barème, seuil du tableau d'honneur.

```sql
INSERT INTO "Decision" (id, domaine, action, libelle, lien, "auteurId", "creeLe", statut,
  "desapprouveeLe", "desapprouveeParId", "motifDesapprobation", annulation, annulee)
VALUES (:id, :domaine, :action, :libelle, :lien, :auteurId, :creeLe, 'APPROUVEE',
  NULL, NULL, NULL, :annulation, :annulee);
```

Modifier

MODIFIER R142. Mettre à jour une décision

Quand : Désapprouver (POST /api/decisions/{id}/desapprouver) : statut DESAPPROUVEE, date, auteur et motif ; complément des données d'annulation juste après la consignation.

```
UPDATE "Decision"
SET domaine = :domaine, action = :action, libelle = :libelle, lien = :lien, "auteurId" = :auteurId,
    statut = :statut, "desapprouveeLe" = :desapprouveeLe, "desapprouveeParId" = :desapprouveeParId,
    "motifDesapprobation" = :motifDesapprobation, annulation = :annulation, annulee = :annulee
WHERE id = :id;
```

Une décision n'est jamais supprimée.

12. Notifications

Messages affichés sous la cloche : le SG est prévenu quand le Directeur désapprouve une de ses décisions ; les professeurs, quand une séquence est ouverte à la saisie.

Tables : `Notification`

Lire

LIRE R143. Les 30 dernières notifications

Quand : Cloche (GET /api/notifications), relue chaque minute.

```
SELECT * FROM "Notification"
WHERE "destinataireId" = :utilisateurId
ORDER BY "creeLe" DESC
LIMIT 30;
```

LIRE R144. Compter les non lues

Quand : Pastille de la cloche.

```
SELECT count(*) FROM "Notification" WHERE "destinataireId" = :utilisateurId AND lue = false;
```

Ajouter

AJOUTER R145. Envoyer une notification

Quand : Une par destinataire : désapprobation (au SG), ouverture d'une séquence (aux professeurs concernés).

```
INSERT INTO "Notification" (id, "destinataireId", titre, message, lien, lue, "creeLe")
VALUES (:id, :destinataireId, :titre, :message, :lien, false, :creeLe);
```

Modifier

MODIFIER R146. Marquer une notification comme lue

Quand : POST /api/notifications/{id}/lue.

```
UPDATE "Notification" SET lue = true WHERE id = :id AND "destinataireId" = :utilisateurId;
```

MODIFIER R147. Tout marquer comme lu

Quand : POST /api/notifications/lues.

```
UPDATE "Notification" SET lue = true WHERE "destinataireId" = :utilisateurId AND lue = false;
```

Les notifications ne sont pas supprimées par l'application ; elles partent avec le compte de leur destinataire.

13. Attestations de présence effective au poste

Délivrées par le Directeur. Chaque attestation garde une copie des informations imprimées (nom, grade, fonction…) : corriger la carrière ensuite ne la modifie pas. Le numéro vient du compteur « attestation-<année> ».

Tables : `Attestation`

Lire

LIRE R148. Lire le registre

Quand : Attestations de présence (GET /api/attestations), les plus récentes d'abord.

```
SELECT a.*, u.*
FROM "Attestation" a
JOIN "Utilisateur" u ON u.id = a."parId"
ORDER BY a."delivreeLe" DESC;
```

LIRE R149. Lire une attestation

Quand : Imprimer une attestation (GET /api/attestations/{id}/pdf).

```
SELECT * FROM "Attestation" WHERE id = :id;
```

Ajouter

AJOUTER R150. Délivrer une attestation

Quand : POST /api/attestations, après « Attribuer le numéro suivant » (voir Élèves) et la mise à jour de la carrière du bénéficiaire.

```
INSERT INTO "Attestation" (id, reference, numero, "anneeLibelle", "utilisateurId", beneficiaire, sexe,
  matricule, grade, fonction, "datePriseService", "acteNomination", signataire, "parId", "delivreeLe")
VALUES (:id, :reference, :numero, :anneeLibelle, :utilisateurId, :beneficiaire, :sexe,
  :matricule, :grade, :fonction, :datePriseService, :acteNomination, :signataire, :parId,
:delivreeLe);
```

Une attestation n'est ni modifiée ni supprimée par l'application ; elle part avec le compte de son bénéficiaire.

14. Suppressions en cascade

Les clés étrangères décident de ce que fait la base quand on supprime une ligne référencée : elle supprime aussi les lignes qui en dépendent (**cascade**), vide le champ qui la désigne (**mise à vide**), ou **refuse** la suppression. L'application vérifie la plupart de ces refus avant d'agir, pour donner un message clair.

On supprime…	La base supprime aussi	La base vide	La base refuse si…
une année	ses séquences, classes, groupes et inscriptions, puis en chaîne matières, notes, compétences, renvois, rattachements	—	—
une classe	ses matières (puis leurs notes et compétences) et ses rattachements aux groupes	—	une inscription la référence
un groupe	ses rattachements aux classes	—	une matière y est rangée
une spécialité	—	—	une classe la porte
une matière	ses notes et ses compétences visées	—	—
une séquence	ses notes et ses compétences visées	—	—
un élève	ses inscriptions, puis leurs notes et leurs renvois	—	—
une inscription	ses notes et ses renvois	—	—
un compte	ses notifications ; les attestations dont il est bénéficiaire	professeur principal des classes, professeur des matières, auteur d'une réintégration, auteur d'une désapprobation	il a saisi des notes, consigné des décisions, renvoyé un élève ou délivré une attestation

> Dans l'application, les suppressions directes sont limitées : spécialité, classe vide, groupe vide, matière (avec confirmation si elle a des notes), note effacée, compétence vidée, barème remplacé, rattachement d'une classe à un groupe. Les élèves, inscriptions, comptes et années ne sont supprimés que lorsque le Directeur désapprouve leur création, et seulement si rien n'en dépend encore.

15. Opérations en plusieurs requêtes

Une action de l'utilisateur déclenche souvent plusieurs requêtes, dans une seule transaction : si l'une échoue, aucune n'est gardée. Quelques exemples, dans l'ordre d'exécution :

Inscrire un élève

1. Lire l'établissement (préfixe des matricules) et la classe.
2. Attribuer le numéro suivant du compteur des matricules.
3. Créer la fiche de l'élève, puis son inscription dans la classe.
4. Consigner la décision dans le journal.

Enregistrer une feuille de notes

1. Lire la matière et la séquence (la saisie doit être ouverte).
2. Vérifier que les élèves sont des inscriptions actives de la classe.
3. Pour chaque élève : lire sa note, puis l'ajouter, la changer ou l'effacer.
4. Lire la compétence visée, puis l'ajouter, la changer ou l'effacer.

Créer une classe

1. Lire l'année et la spécialité.
2. Lire les classes du même niveau et de la même spécialité, et le plus grand numéro.
3. Créer la classe, puis renommer les classes voisines si elles prennent une lettre.
4. Consigner la décision.

Supprimer une classe

1. Lire la classe et compter ses inscriptions (il ne doit y en avoir aucune).
2. Lire ses matières et les supprimer une à une.
3. Supprimer la classe, puis renommer les classes voisines.
4. Consigner la décision.

Ouvrir une séquence

1. Lire la séquence et la mettre à jour (saisie ouverte).
2. Consigner la décision.
3. Lire les professeurs actifs qui enseignent dans l'année et leur envoyer une notification.

Créer une année en reprenant l'organisation

1. Créer l'année et ses six séquences ; si elle devient l'année en cours, désactiver les autres.
2. Lire les groupes et les classes de l'année copiée ; créer les groupes, les classes, leurs rattachements et les matières.
3. Consigner la décision.

Délivrer une attestation

1. Mettre à jour la carrière du bénéficiaire (requête précédente, depuis la même fenêtre).
2. Lire l'année en cours, l'établissement, le bénéficiaire et le Directeur.

3. Attribuer le numéro suivant du compteur « attestation-<année> ».
4. Enregistrer l'attestation.

Désapprouver une décision

1. Lire la décision (moins de 4 semaines, encore approuvée).
2. Défaire l'action à partir des données gardées dans la colonne `annulation` : suppression de ce qui a été créé, remise des valeurs précédentes, recréation de ce qui a été supprimé ; les vérifications (notes saisies, élèves inscrits) peuvent tout refuser.
3. Mettre à jour la décision (désapprouvée, motif, auteur, date).
4. Envoyer une notification au Surveillant Général.

16. Démarrage : migrations et amorçage

Migrations Flyway

Au démarrage, Flyway applique les migrations qui manquent, dans l'ordre, et les note dans `flyway_schema_history`. Le schéma n'est jamais modifié par Hibernate. Les quatre premières migrations viennent de l'ancienne API Node (Prisma) ; sur une base qu'elle avait créée, Flyway part de la version 4 sans les rejouer.

Fichier	Contenu
`V1__init.sql`	Tables d'origine, énumérations, clés étrangères et index
`V2__specialites_decisions.sql`	Spécialités et numéro des classes (lettres A, B…), journal des décisions
`V3__annulation_decisions.sql`	Données d'annulation des décisions (JSON)
`V4__notifications.sql`	Notifications
`V5__entete_carriere_attestations_tableau_honneur.sql`	Sigle anglais et seuil du tableau d'honneur, carrière des personnels, attestations

Amorçage d'une base vide

Lancée avec l'option `--amorcer`, l'API remplit une base vide, en sautant ce qui existe déjà :

1. l'établissement (« Savoir si l'établissement existe », puis « Créer l'établissement ») ;
2. les barèmes par défaut (« Compter les barèmes », puis « Ajouter une mention » pour chacune) ;
3. l'année en cours et ses six séquences (« Compter les années », « Créer une année », « Créer une séquence ») ;
4. les comptes du Directeur et du Surveillant Général (« Savoir si un compte de ce rôle existe », puis « Créer un compte »).